

Python 程式語言

Lecture 12: Introduction to Machine Learning with Applications

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
 - Introduction to Python and Development Environment Setup
 - Basic Syntax and Flow Control
 - Data Structures and Collections
- Advanced Programming
 - Modular and Functional Programming
 - File Handling and Exception Handling
 - Object-Oriented Programming
 - Algorithm Implementation in Python
- User Interface Development
 - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
 - Data Analysis Fundamentals
 - Scientific Computing Applications
 - Data Visualization
- AI & ML Applications
 - Introduction to Machine Learning with Applications
 - AI-Related Library in Python Applications

Course Objectives

- Understand fundamental machine learning concepts
- Use scikit-learn for model development
- Implement classification and regression models
- Evaluate and optimize ML models

Machine Learning Concepts

Algorithm Paradigms Comparison

- Three Approaches to Problem Solving
 - In computer science, we have three main algorithmic paradigms for problem-solving
 - Problem $\rightarrow$ Solution
 - Method 1: Exact Algorithms
 - Method 2: Heuristic Algorithms
 - Method 3: Machine Learning Algorithms

1. Exact Algorithms

- Algorithms that guarantee finding the optimal solution through complete enumeration or mathematical reasoning
- Characteristics
 - **Guaranteed correctness:** Always produces the correct/optimal answer
 - **Deterministic:** Same input always produces same output
 - **Computationally expensive:** May be too slow for large or complex problems

Algorithm Paradigms Comparison

2. Heuristic Algorithms

- Algorithms that use practical methods to find "good enough" solutions quickly, without guaranteeing optimality
- Characteristics
 - Fast execution: Much faster than exact algorithms
 - Scalable: Works well with large problems
 - No optimality guarantee: Solution may be suboptimal
 - Problem-specific: Often designed for specific problem types

3. Machine Learning Algorithms

- Algorithms that learn patterns from data to make predictions or decisions, improving through experience
- Characteristics
 - Learns from data: Automatically discovers patterns
 - Handles complexity: Works with high-dimensional, noisy data
 - Generalizes: Can handle unseen cases
 - Requires training data: Performance depends on data quality and quantity
 - Less interpretable: Often a "black box"

Detailed Comparison Table

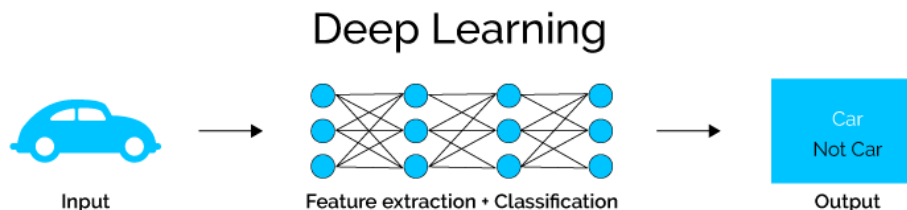
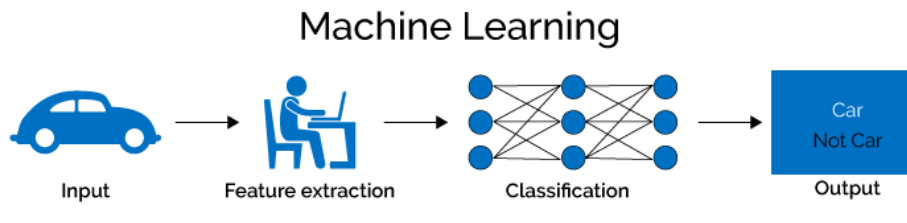
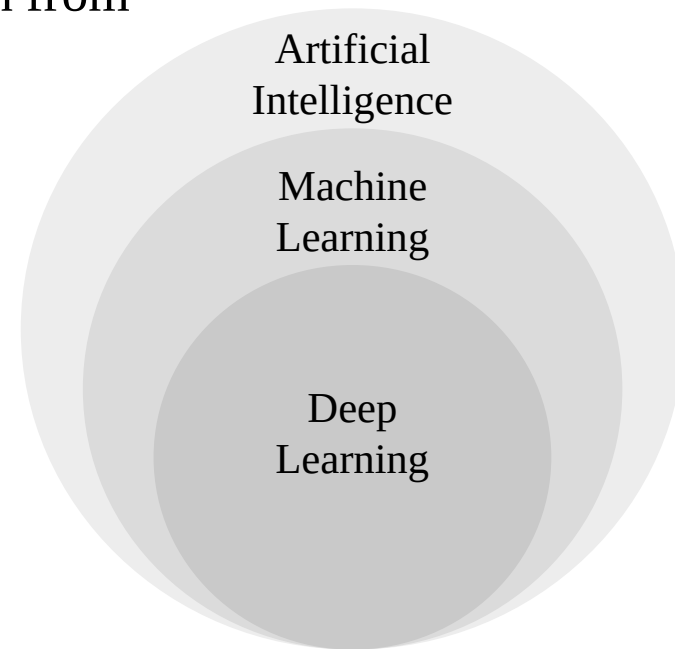
Aspect	Exact Algorithms	Heuristic Algorithms	ML Algorithms
Solution Quality	Optimal	Good enough	Depends on data
Speed	Slow for complex problems	Fast	Fast (after training)
Scalability	Poor	Good	Excellent
Requires Data	No	No	Yes (lots!)
Adaptability	Fixed	Fixed	Learns & improves
Interpretability	High	Medium	Low
Development Time	Medium	Medium-High	High (data prep)
Uncertainty Handling	Poor	Poor	Good

- Quick Reference Guide

If your problem has...	Use...
Small dataset, need perfect answer	Exact Algorithm
Large dataset, speed critical, good-enough OK	Heuristic Algorithm
Huge dataset, complex patterns, can tolerate errors	Machine Learning
Mission-critical + lots of data	Hybrid Approach

What is Machine Learning?

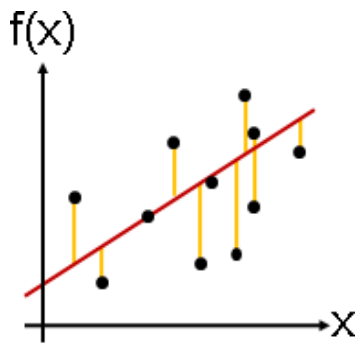
- Definition
 - Machine Learning is a subset of Artificial Intelligence that enables computers to learn from data without being explicitly programmed
- Key Characteristics
 - Learning from data: **Algorithms improve performance through experience**
 - Pattern recognition: Identify **patterns** and relationships in data
 - Prediction making: Make decisions or predictions based on learned patterns



Types of Machine Learning

- Supervised Learning
 - Training data includes **both input features and correct output labels**
 - Examples: Email spam detection, House price prediction, Image classification
- Semi-Supervised Learning
 - Training data includes **BOTH labeled AND unlabeled examples**
 - Small labeled dataset + Large unlabeled dataset → Model
- Unsupervised Learning
 - Training data **only includes input features, no labels**
 - Examples: Customer segmentation, Anomaly detection, Dimensionality reduction
- Reinforcement Learning
 - **Agent learns through interaction with environment**
 - Examples: Game playing AI, Robotics control, Autonomous vehicles

Machine Learning Applications

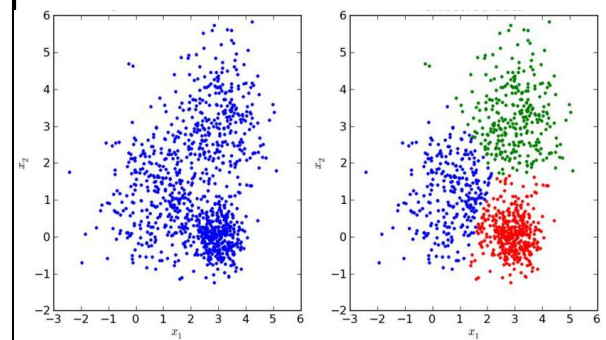
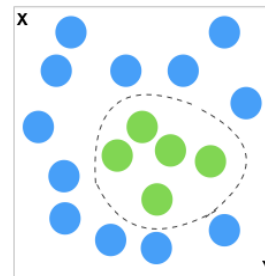
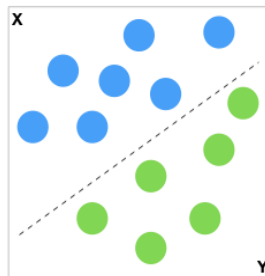


Regression

Numerical Prediction,
Survival Analysis,
Distribution Prediction

Classification

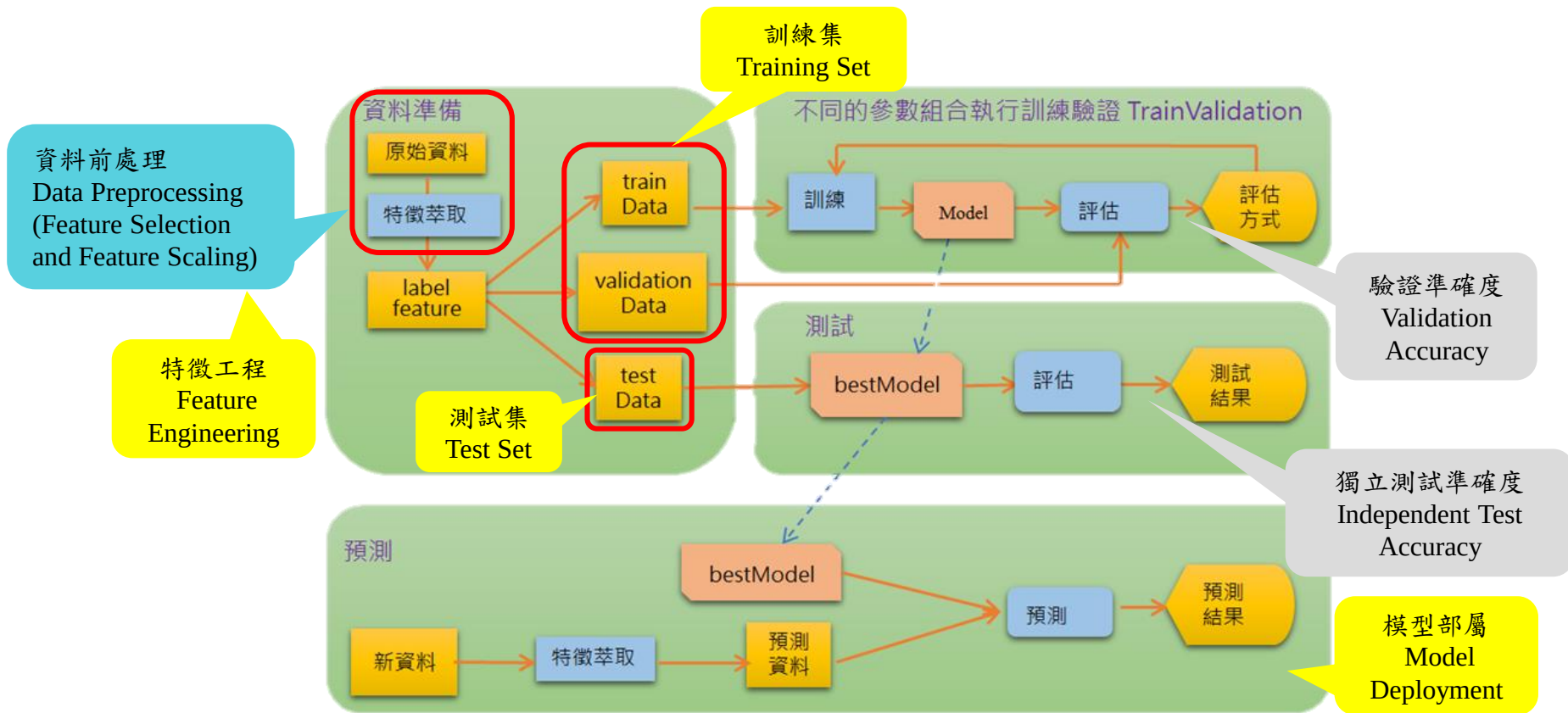
Binary Classification,
Multi-class Classification,
Decision Support



Clustering

Data Grouping, Anomaly
Detection

Machine Learning Workflow



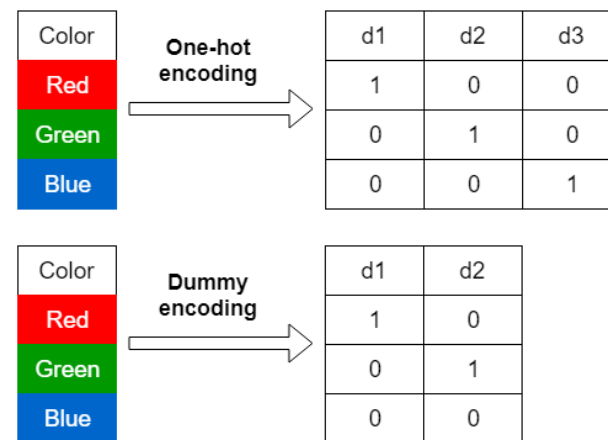
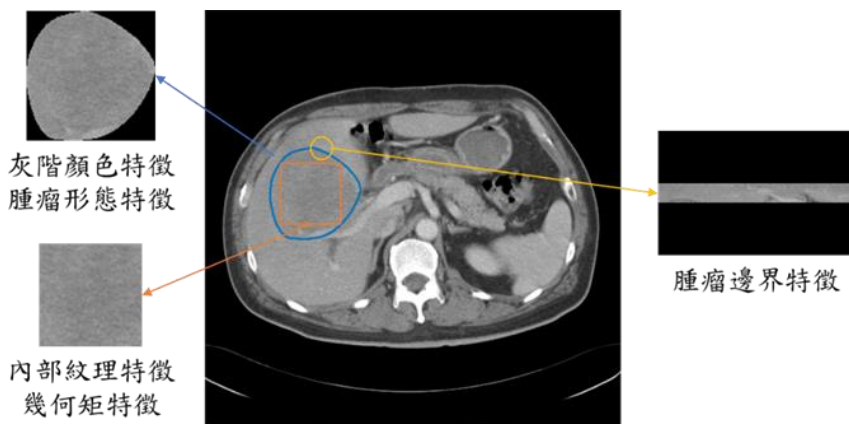
Data Preparation - ML Data Format

- Core Concepts
 - Features: Attributes used to describe the data
 - Labels: The answers we want to predict
- Example: Fruit Classifier
 - Problem: How can machines distinguish between oranges and apples?
 - Possible Distinguishing Features: Color? Surface Texture? Taste?
- Building Dataset
 - Selected Features: Weight, Surface Texture
 - Define Labels: Fruit Type
(This is the answer we want the machine to learn to predict)
- Data Encoding
 - Why Encoding? Computers can only process numbers; text must be converted to numerical values
 - Encoding Rules

Category	Original	Encoded Value
Surface Texture	Wrinkled	1
	Smooth	0
Fruit Type	Orange	0
	Apple	1

Feature Extraction, Encoding, and Label Types

- Feature Extraction
 - Image Feature Extraction: Extracting meaningful features from raw data
 - Encoding Methods: One-Hot Encoding, Dummy Encoding
 - Secondary Features: New features derived from original features
- Label Types
 - Categorical Labels: Discrete category values (Classification)
 - Continuous Numerical Labels: Continuous numerical range (Regression)



Training Data vs Testing Data

- Why Split Data?
- Problem: If we test on training data, we can't evaluate real performance
- Solution: Split data into training and testing sets

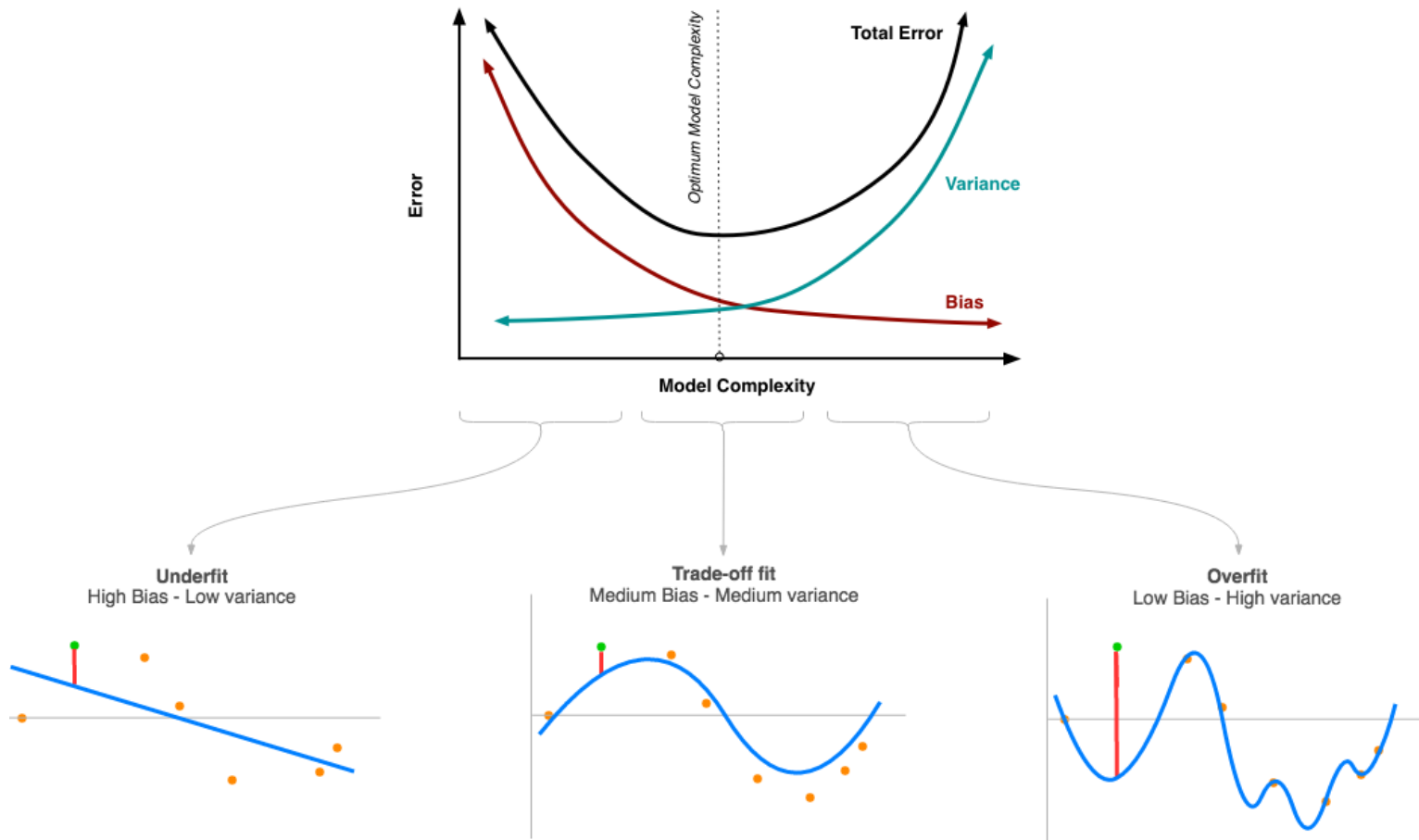
```
from sklearn.model_selection import train_test_split
# Example: Split data
# 80% for training, 20% for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, # Features
    y, # Labels
    test_size=0.2, # 20% for testing 20%
    random_state=42 # For reproducibility
)
print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")
```

Overfitting vs Underfitting

- Underfitting
 - Model is too simple and doesn't capture data patterns
 - Poor performance on training data
 - Poor performance on test data
 - Solution: Use more complex model or add features
- Overfitting
 - Model learns training data too well, including noise
 - Excellent performance on training data
 - Poor performance on test data
 - Solution: Regularization, more data, or simpler model

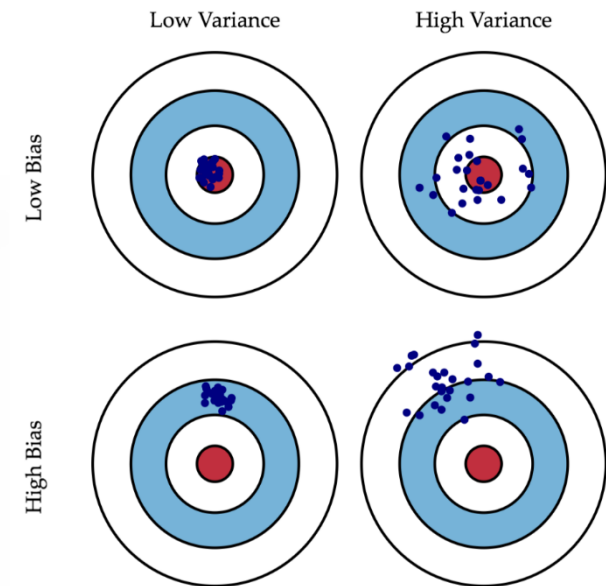
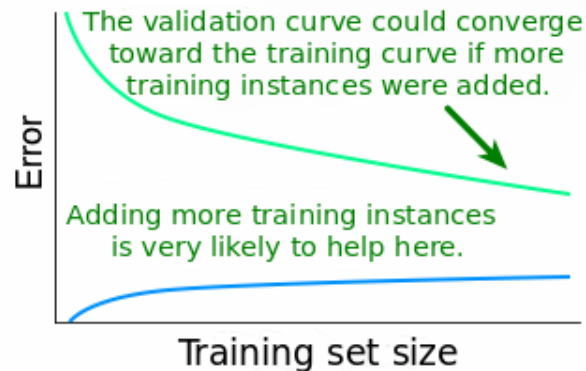
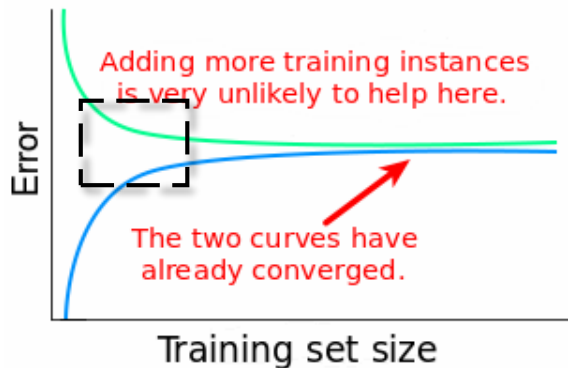
Good Fit: The Sweet Spot

- Goal: Find the balance where test error is minimized



Learning Curve Analysis

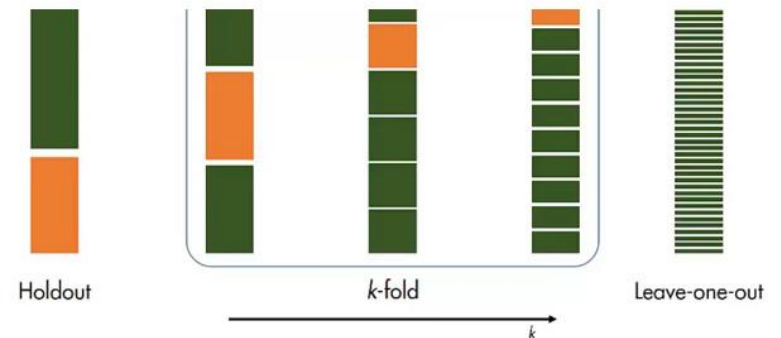
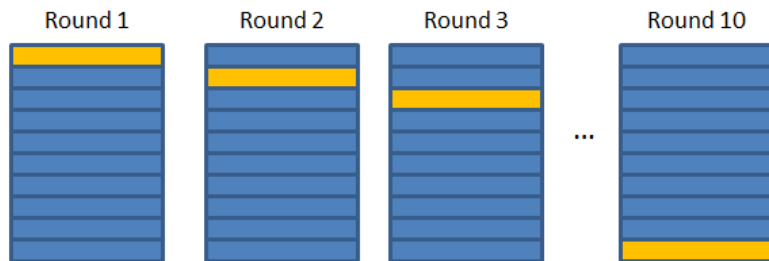
- Model Performance vs Training Set Size
 - Small Dataset: Low Bias, High Variance → Overfitting
 - Medium Dataset: Moderate Bias, Moderate Variance → Good Fit
 - Large Dataset: Moderate Bias, Low Variance → Stable Prediction
- Bias-Variance Trade-off
 - Bias
 - Systematic error of the model
 - Gap between model assumptions and reality
 - High bias → Underfitting
 - Variance
 - Model's sensitivity to training data
 - Variation across different training sets
 - High variance → Overfitting



Cross-Validation

- K-Fold Cross-Validation
 - Instead of single train-test split, use multiple splits for robust evaluation
 - Benefits
 - More reliable performance estimate
 - Better use of limited data
 - Reduces impact of data splitting
- Leave-One-Out Cross-Validation (LOOCV)

Validation Set
Training Set



scikit-learn Basics

Introduction to scikit-learn

- What is scikit-learn?
 - scikit-learn is Python's most popular machine learning library
- Key Features
 - Comprehensive ML algorithms
 - Consistent API design
 - Excellent documentation
 - Integration with NumPy/Pandas
 - Well-maintained and stable
- Installation
 - `pip install scikit-learn`
 - `conda install scikit-learn`

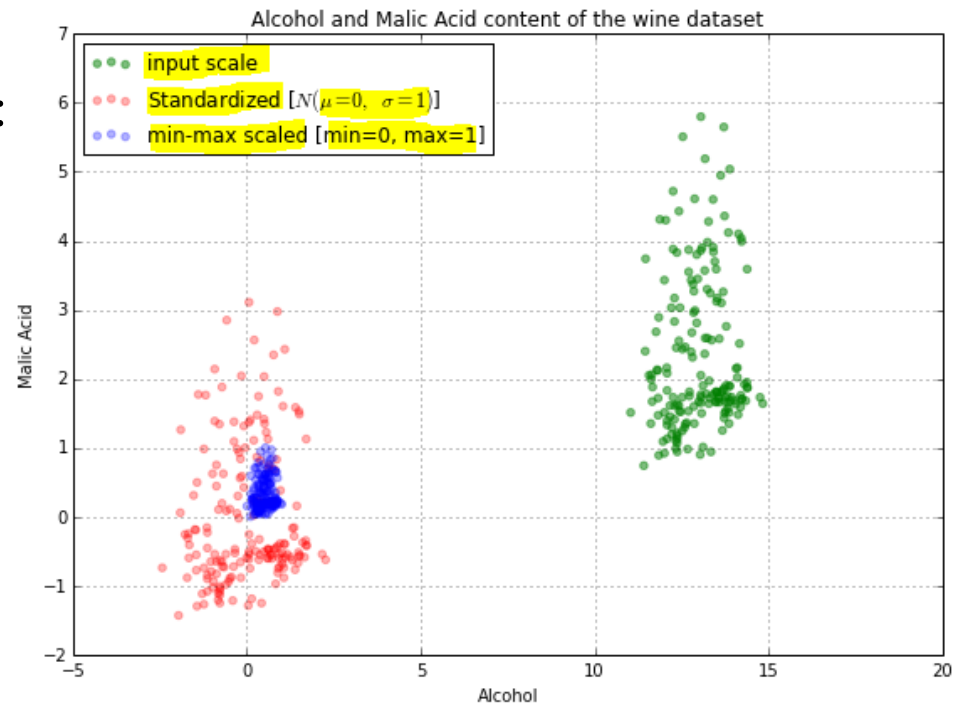
Data Preprocessing

- Loading Sample Datasets

```
from sklearn.datasets import load_iris, load_diabetes
import pandas as pd
# Load iris dataset
iris = load_iris() # Convert to DataFrame
df = pd.DataFrame(
    data=iris.data,
    columns=iris.feature_names
)
df['target'] = iris.target
df['species'] = df['target'].map(
    {0: 'setosa', 1: 'versicolor', 2: 'virginica'}
)
print("Iris Dataset:")
print(df.head())
print(f"\nShape: {df.shape}")
print(f"Features: {iris.feature_names}")
print(f"Classes: {iris.target_names}")
```

Feature Scaling

- Why Scale Features?
 - Different features often have different scales, which can impact model performance
- Standardization (StandardScaler):
When features follow normal distribution
 - $$X_{change} = \frac{X - \mu}{\sigma}$$
- Normalization (MinMaxScaler):
When you need bounded range
 - $$X_{change} = \frac{X - X_{min}}{X_{max} - X_{min}}$$



Feature Scaling

- StandardScaler

```
from sklearn.preprocessing import StandardScaler
import numpy as np

# Example data with different scales
data = np.array([
    [1, 100, 0.5], # Feature 1: 1-10
    [5, 500, 2.3], # Feature 2: 100-1000
    [10, 1000, 4.8] # Feature 3: 0.1-5.0
])

# Create and fit scaler
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

print("Original data:")
print(data)
print("\nScaled data (mean=0, std=1):")
print(scaled_data)
print(f"\nMean: {scaled_data.mean(axis=0)}")
print(f"\nStd: {scaled_data.std(axis=0)}")
```

- MinMaxScaler

```
from sklearn.preprocessing import MinMaxScaler

# Scale to range [0, 1]
minmax_scaler = MinMaxScaler()
minmax_scaled = minmax_scaler.fit_transform(data)

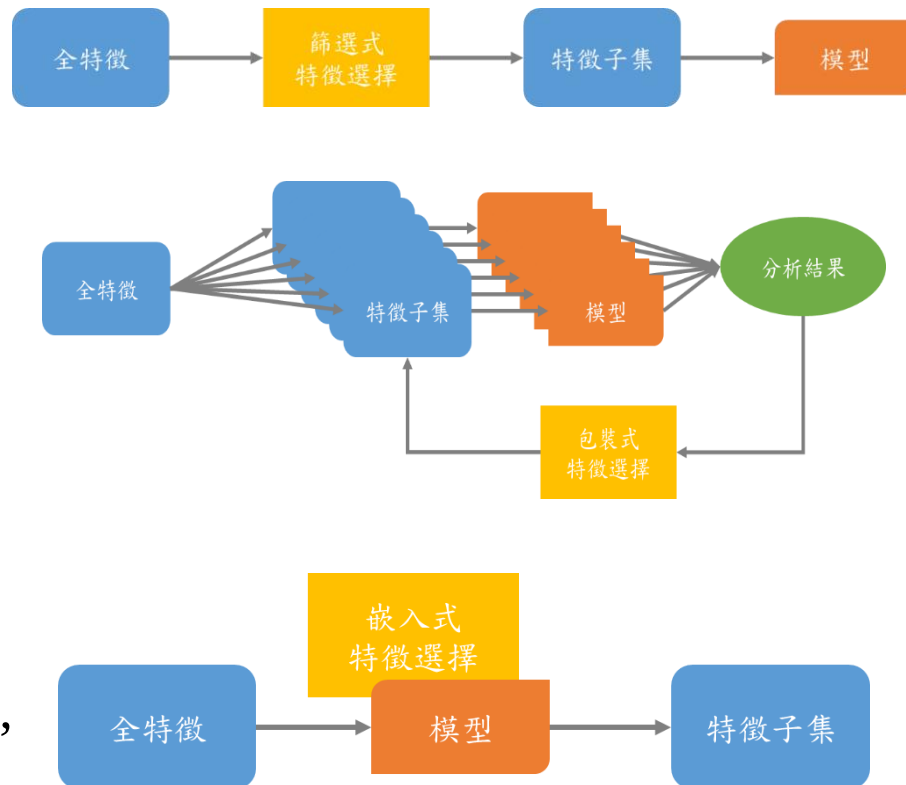
print("MinMax scaled data (range 0-1):")
print(minmax_scaled)
print(f"\nMin: {minmax_scaled.min(axis=0)}")
print(f"\nMax: {minmax_scaled.max(axis=0)}")

# Custom range [0, 10]
custom_scaler = MinMaxScaler(feature_range=(0, 10))
custom_scaled = custom_scaler.fit_transform(data)

print("\nCustom range [0, 10]:")
print(custom_scaled)
```

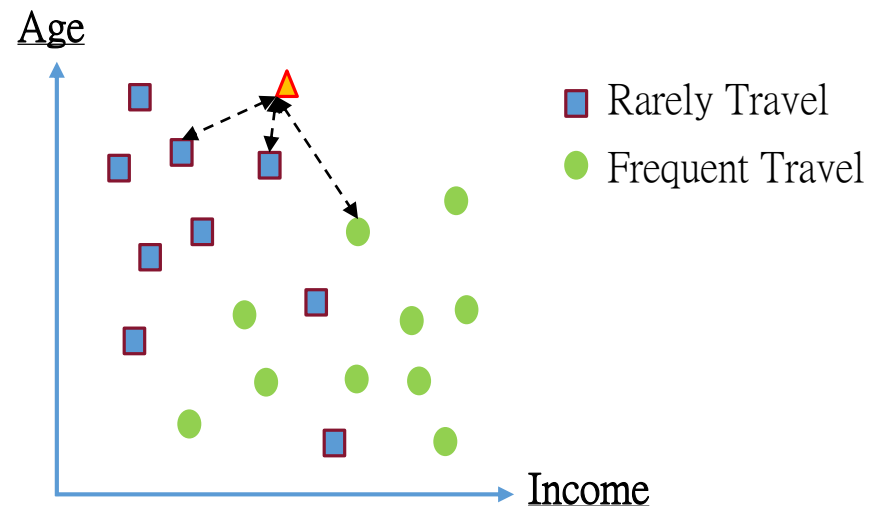
Feature Selection

- Why Feature Selection?
- Benefits
 - Reduces overfitting
 - Improves model performance
 - Reduces training time
 - Improves interpretability
- Type
 - Filter (Fish-Score, Relief-F, Correlation Feature Selection (CFS))
 - Wrapper (Forward Selection, Backward Elimination)
 - Embedded (Lasso Regression, Ridge Regression, Elastic Net)



Classification: K-Nearest Neighbors (KNN)

- How KNN Works
 - Classify based on the K nearest data points in feature space
- Steps:
 - Calculate distance to all training samples
 - Find K nearest neighbors
 - Vote for the most common class
- Considerations:
 - Small K: More sensitive to noise
 - Large K: More stable but less precise
 - Odd K: Avoids ties in binary classification



Classification: K-Nearest Neighbors (KNN)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
```

```
# Create and train KNN model
```

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
```

```
# Load data
```

```
iris = load_iris()
X, y = iris.data, iris.target
```

```
# Make predictions
```

```
y_pred = knn.predict(X_test)
```

```
# Check accuracy
```

```
accuracy = knn.score(X_test, y_test)
print(f"KNN Accuracy: {accuracy:.3f}")
```

```
# Split data
```

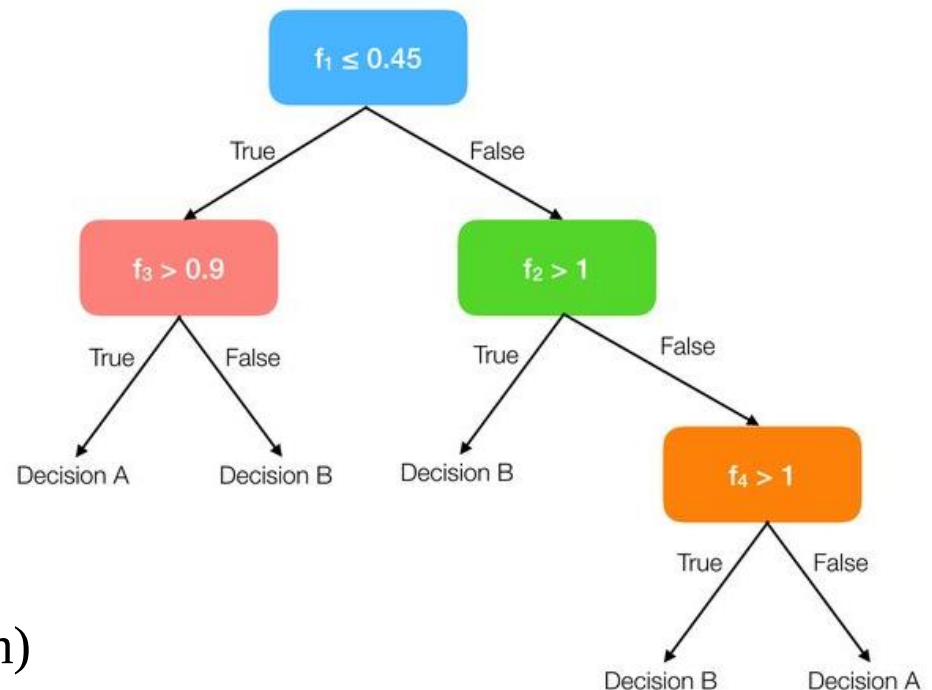
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Predict single sample
```

```
sample = [[5.1, 3.5, 1.4, 0.2]]
prediction = knn.predict(sample)
print(f"Predicted class: {iris.target_names[prediction[0]]}")
```

Decision Tree Classifier

- How Decision Trees Work
 - Decision trees make decisions by asking a series of questions about the features
- Advantages
 - Easy to understand and interpret
 - Requires little data preparation
 - Handles both numerical and categorical data
- Disadvantages
 - Can easily overfit
 - Sensitive to small variations in data
 - Greedy algorithm (may not find global optimum)



Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Create decision tree
dt_classifier = DecisionTreeClassifier(
    max_depth=3,      # Limit tree depth
    random_state=42
)

# Train model
dt_classifier.fit(X_train, y_train)

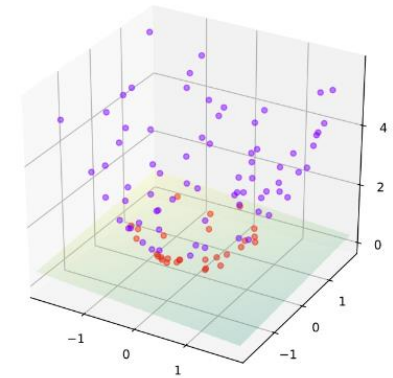
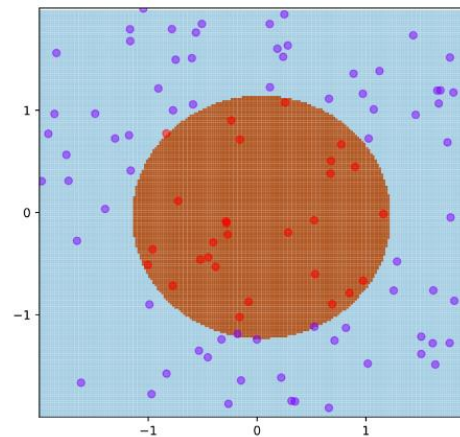
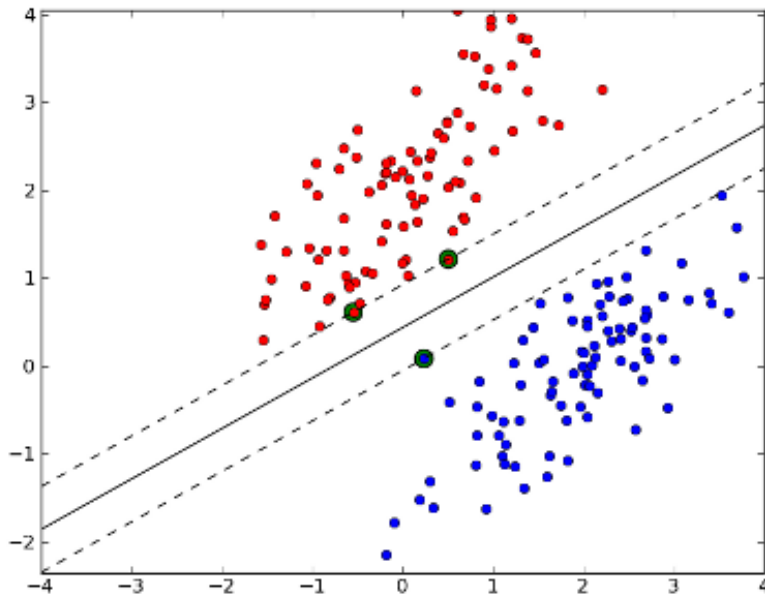
# Evaluate
train_acc = dt_classifier.score(X_train, y_train)
test_acc = dt_classifier.score(X_test, y_test)

print(f"Decision Tree Performance:")
print(f"Training Accuracy: {train_acc:.3f}")
print(f"Testing Accuracy: {test_acc:.3f}")

# Visualize tree
plt.figure(figsize=(15, 10))
plot_tree( dt_classifier, feature_names=iris.feature_names, class_names=iris.target_names, filled=True, rounded=True )
plt.title("Decision Tree Visualization ")
plt.show()
```

Support Vector Machine (SVM)

- SVM Concept
 - Find the optimal hyperplane that best separates different classes

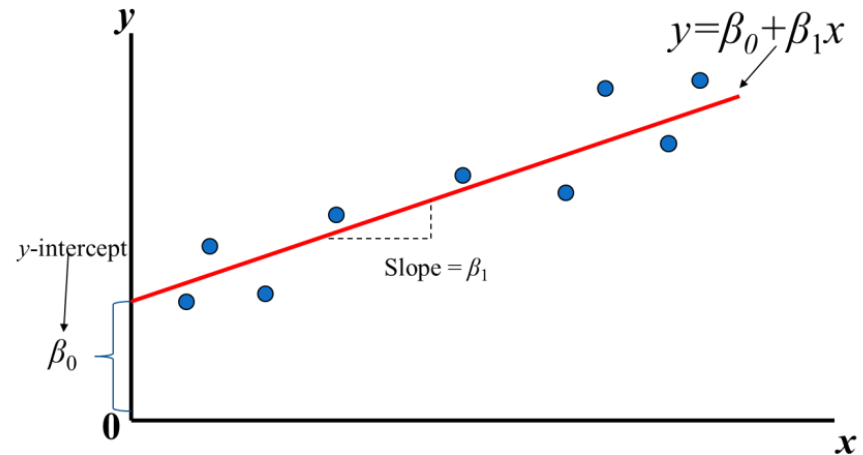


Support Vector Machine (SVM)

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
# Scale features (important for SVM!)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Create SVM classifier
svm_classifier = SVC(
    kernel='rbf', # Radial basis function kernel
    C=1.0, # Regularization parameter
    random_state=42
)
# Train model
svm_classifier.fit(X_train_scaled, y_train)
# Evaluate
train_acc = svm_classifier.score(X_train_scaled, y_train)
test_acc = svm_classifier.score(X_test_scaled, y_test)
print(f"SVM Performance:")
print(f"Training Accuracy: {train_acc:.3f}")
print(f"Testing Accuracy: {test_acc:.3f}")
print(f"Number of support vectors: {svm_classifier.n_support_}")
```

Regression: Linear Regression

- Understanding R^2 Score
 - R^2 Score measures how well the model explains variance in the data
 - $R^2 = 1$: Perfect prediction
 - $R^2 = 0$: Model no better than mean
 - $R^2 < 0$: Model worse than mean



Regression: Linear Regression

```
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_diabetes
import numpy as np
```

Load diabetes dataset

```
diabetes = load_diabetes()
```

```
X = diabetes.data[:, np.newaxis, 2] # Use only one feature
```

```
y = diabetes.target
```

Split data

```
X_train, X_test, y_train, y_test = train_test_split( X, y,
test_size=0.2, random_state=42 )
```

Create and train model

```
linear_reg = LinearRegression()
```

```
linear_reg.fit(X_train, y_train)
```

Make predictions

```
y_pred = linear_reg.predict(X_test)
```

Model parameters

```
print(f"Coefficient (slope): {linear_reg.coef_[0]:.2f}")
```

```
print(f"Intercept: {linear_reg.intercept_:.2f}")
```

Evaluate

```
from sklearn.metrics import mean_squared_error,
r2_score
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print(f"\nModel Performance:")
```

```
print(f"Mean Squared Error: {mse:.2f}")
```

```
print(f"R2 Score: {r2:.3f}")
```

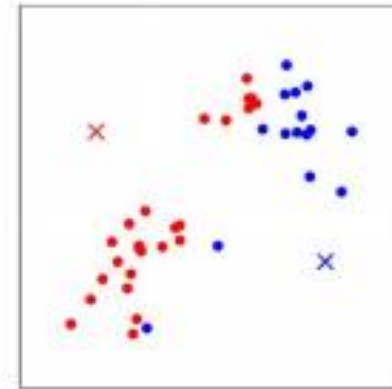

Clustering: K-Means



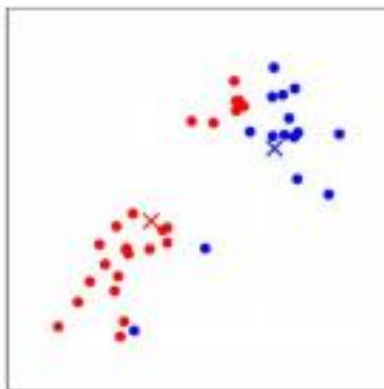
(a)



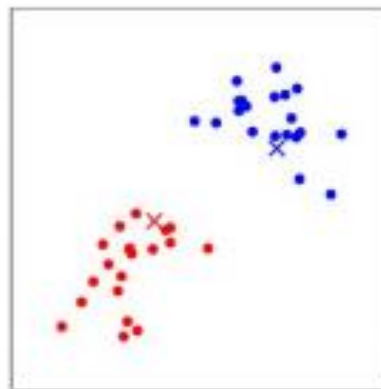
(b)



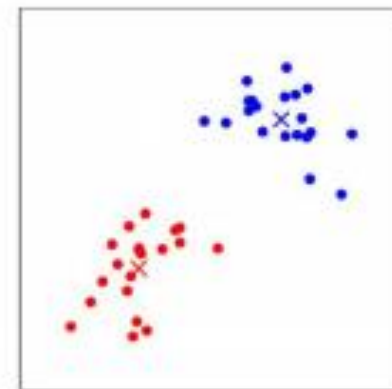
(c)



(d)



(e)



(f)

Clustering: K-Means

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

```
# Generate sample data
```

```
X_cluster, y_true = make_blobs(
    n_samples=300,
    centers=4,
    cluster_std=0.60,
    random_state=42
)
```

```
# Create K-Means model
```

```
kmeans = KMeans(
    n_clusters=4, # Number of clusters
    random_state=42,
    n_init=10
)
```

```
# Fit and predict
```

```
y_kmeans = kmeans.fit_predict(X_cluster)
```

```
# Get cluster centers
```

```
centers = kmeans.cluster_centers_
```

```
print(f"Cluster Centers:")
```

```
print(centers)
```

```
print(f"\nInertia (within-cluster sum of squares):  
{kmeans.inertia_:.2f}")
```

Visualizing K-Means Clustering

Visualize clusters

```
plt.figure(figsize=(12, 5))
```

Original labels (if available)

```
plt.subplot(1, 2, 1)
```

```
plt.scatter(X_cluster[:, 0], X_cluster[:, 1], c=y_true,  
            cmap='viridis', alpha=0.6)
```

```
plt.title('True Labels')
```

```
plt.xlabel('Feature 1')
```

```
plt.ylabel('Feature 2')
```

K-Means predicted labels

```
plt.subplot(1, 2, 2)
```

```
plt.scatter(X_cluster[:, 0], X_cluster[:, 1], c=y_kmeans,  
            cmap='viridis', alpha=0.6)
```

```
plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='X',  
            s=200, edgecolors='black', linewidth=2,  
            label='Centroids')
```

```
plt.title('K-Means Clustering')
```

```
plt.xlabel('Feature 1')
```

```
plt.ylabel('Feature 2')
```

```
plt.legend()
```

```
plt.tight_layout()
```

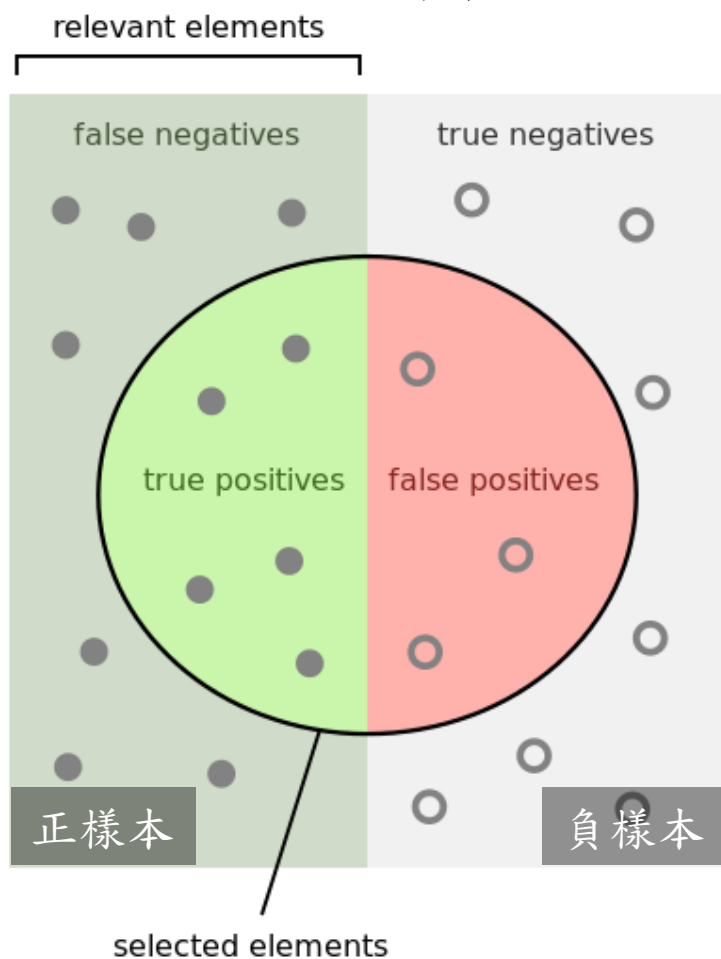
```
plt.show()
```

Model Evaluation and Optimization

Confusion Matrix

- Understanding Classification Results

二分類圖解



	模型預測為正 (predicted positive)	模型預測為負 (predicted negative)
實際分類為正 (actual positive)	True positive (TP)	False negative (FN)
實際分類為負 (actual negative)	False positive (FP)	True negative (TN)

True / False：模型預測結果與真實狀況**契合與否**

Positive / Negative：模型**預測**為正/模型預測為負

Classification Metrics

- Key Performance Metrics

- Accuracy** = $\frac{TP+TN}{all}$
- Positive predictive value (PPV) = $\frac{TP}{TP+FP} = \frac{TP}{pred+}$ = **Precision**
- False discovery rate (FDR) = $\frac{FP}{TP+FP} = \frac{FP}{pred+}$
- False omission rate (FOR) = $\frac{FN}{FN+TN} = \frac{FN}{pred-}$
- Negative predictive value (NPV) = $\frac{FN}{FN+TN} = \frac{FN}{pred-}$
- True positive rate (TPR) = $\frac{TP}{TP+FN} = \frac{TP}{actual+}$ = **Sensitivity** = **Recall**
- False positive rate (FPR) = $\frac{FP}{FP+TN} = \frac{FP}{actual-}$ = Fall-out

	(predicted +)	(predicted -)
(actual +)	True positive (TP)	False negative (FN)
(actual -)	False positive (FP)	True negative (TN)

Classification Metrics

- False negative rate (FNR) = $\frac{FN}{TP+FN} = \frac{FN}{actual+}$ = Miss rate
- True negative rate (TNR) = $\frac{TN}{FP+TN} = \frac{TN}{actual-}$ = **Specificity**
- Positive likelihood ratio (LR+) = $\frac{TPR}{FPR} = \frac{TP(FP+TN)}{FP(TP+FN)} = \frac{TP*actual-}{FP*actual+}$
- Negative likelihood ratio (LR-) = $\frac{FNR}{TNR} = \frac{FN(FP+TN)}{TN(TP+FN)} = \frac{FN*actual-}{TN*actual+}$
- Diagnosis odds ratio (DOR) = $\frac{LR+}{LR-} = \frac{TP*TN}{FP*FN}$
- F1-score** = $2 * \frac{Precision * Recall}{Precision + Recall} = \frac{2*TP}{2*TP+FP+FN} = 2 * \frac{TP}{(pred+)+(actual+)}$
- G-measure = $\sqrt{Precision * Recall} = \frac{TP}{\sqrt{(TP+FP)(TP+FN)}} = \frac{TP}{\sqrt{(pred+)(actual+)}}$
- MCC* = $\frac{TP*TN-FP*FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$

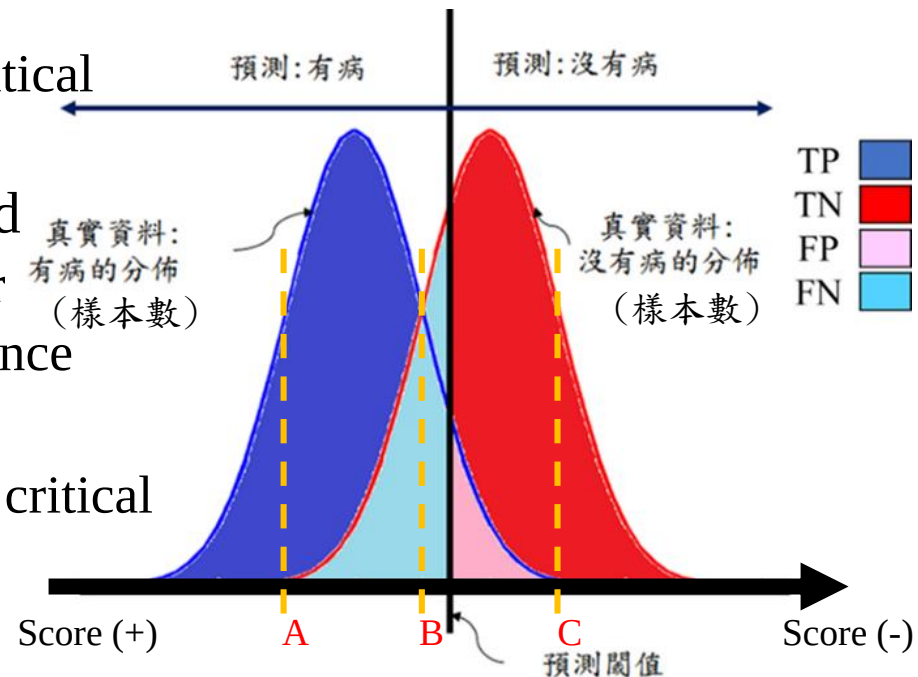
	(predicted +)	(predicted -)
(actual +)	True positive (TP)	False negative (FN)
(actual -)	False positive (FP)	True negative (TN)

When to Use Which Metric?

- Scenario-Based Selection
 - Use Accuracy when:
 - Classes are balanced
 - All errors are equally important
 - Use Precision when:
 - False positives are costly
 - Example: Spam detection (don't want to mark important emails as spam)
 - Use Recall when:
 - False negatives are costly
 - Example: Disease diagnosis (don't want to miss sick patients)
 - Use F1-Score when:
 - Need balance between precision and recall
 - Classes are imbalanced

Classification Results with Threshold Variation

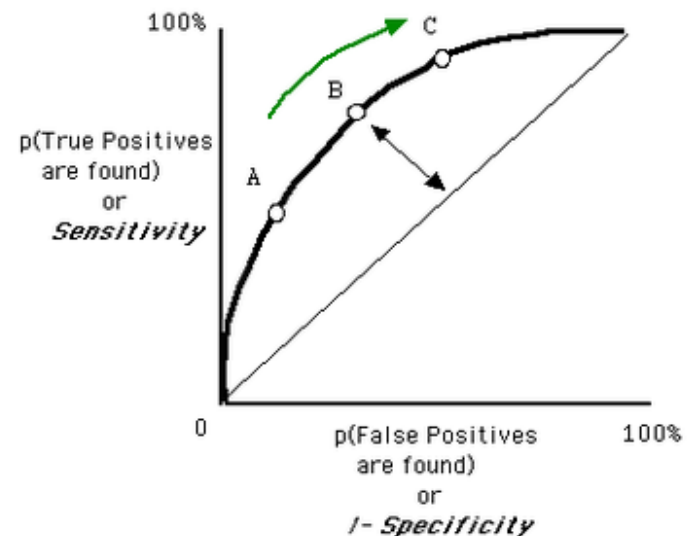
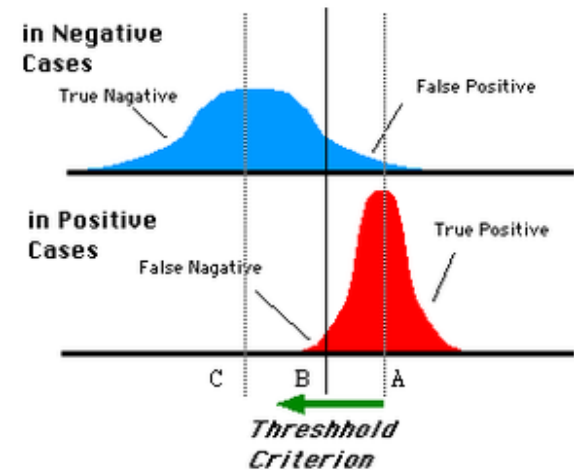
- Classifier Output and Threshold
 - Classifier Output: Usually a probability or score value (Range: 0 - 1 or $-\infty$ - $+\infty$)
 - Threshold: Cutoff point for final classification (Default: 0.5 or 0)
- Three Threshold Strategies
 - Score A: High Threshold
 - Avoiding false alarms is critical
 - Minimum FN
 - Score B: Balanced Threshold
 - FP and FN costs are similar
 - Need best overall performance
 - Score C: Low Threshold
 - Avoiding false negatives is critical
 - Minimum FP



ROC Curve & AUC

- ROC (Receiver Operating Characteristic)
 - Only for binary classification
 - Evaluate model performance and select optimal threshold
 - Principle: Plot the performance of the **same model** at **different thresholds**
 - FPR (False Positive Rate): Proportion of actual negatives incorrectly identified as positive
 - $$\text{FPR} = \frac{FP}{FP+TN}$$
 - TPR (True Positive Rate): Proportion of actual positives correctly identified
 - $$\text{TPR} = \frac{TP}{TP+FN}$$

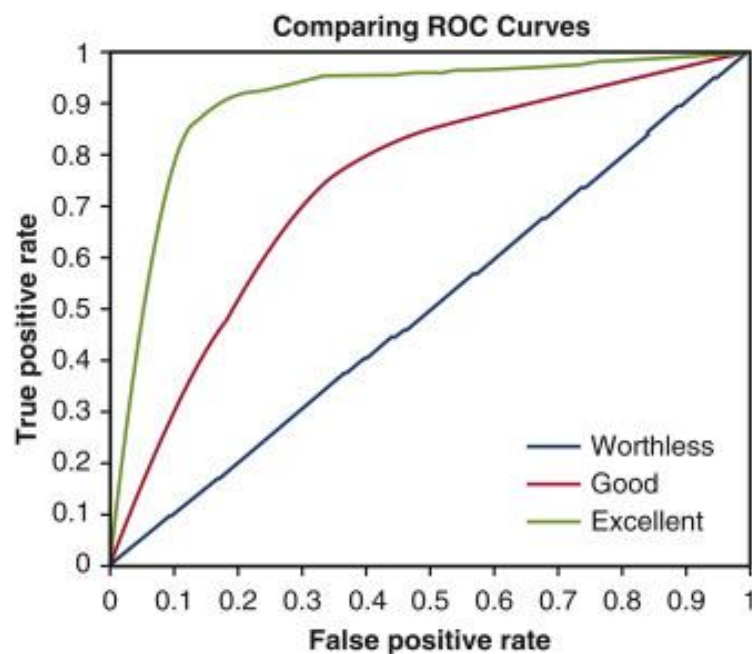
Distributions of the Observed signal strength



ROC curve

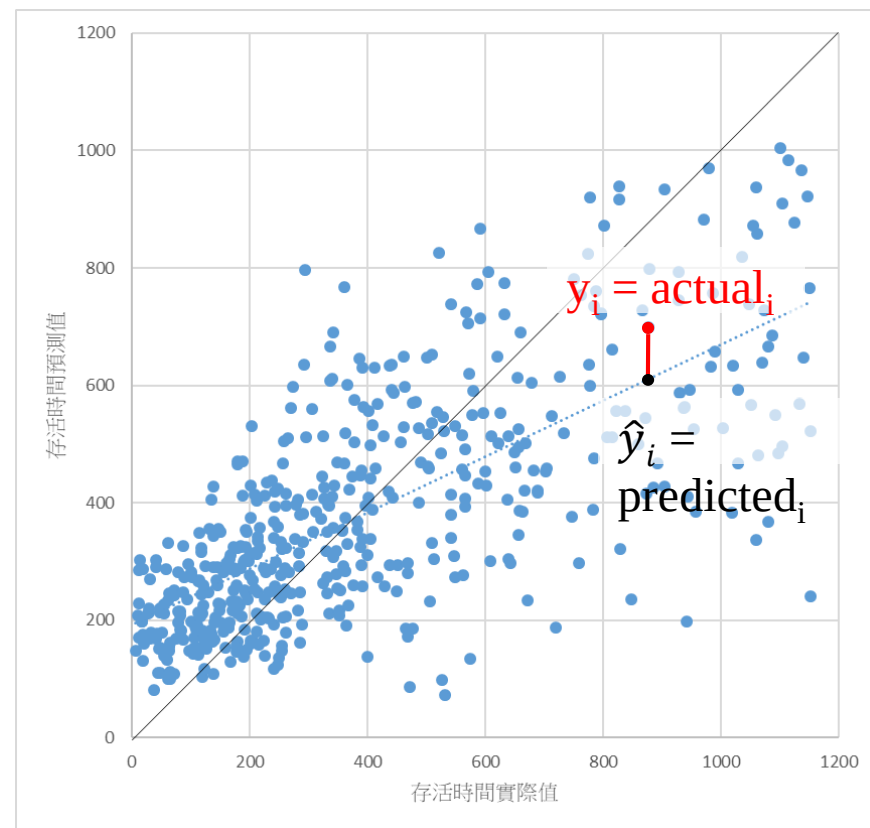
- Model Discrimination Assessment Metric
- Meaning of Curve Position
 - Closer to top-left corner (0, 1) → Better model
 - Closer to diagonal line → Lower discrimination
- Practical Guidelines

AUC Range	Model Quality	Practical Application
0.9 - 1.0	Excellent	Ready to use
0.8 - 0.9	Good	Good performance
0.7 - 0.8	Fair	Acceptable
0.6 - 0.7	Poor	Needs improvement
0.5 - 0.6	Fail	Nearly useless



Regression Metrics

- Mean Absolute Error (MAE)
 - Average absolute difference (less sensitive to outliers)
- Mean Squared Error (MSE)
 - Average squared difference (penalizes large errors)
- Root Mean Squared Error (RMSE)
 - Square root of MSE (same unit as target)
- R^2 Score
 - Proportion of variance explained (0 to 1, higher is better)



Model Comparison

- Comparing Multiple Algorithms

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier

# Define models to compare
models = {
    'KNN': KNeighborsClassifier(n_neighbors=3),
    'Decision Tree': DecisionTreeClassifier(max_depth=3,
random_state=42),
    'SVM': SVC(kernel='rbf', random_state=42),
    'Logistic Regression':
LogisticRegression(random_state=42, max_iter=200),
    'Naive Bayes': GaussianNB(),
    'Random Forest':
RandomForestClassifier(n_estimators=100,
random_state=42)
}

# Train and evaluate each model
results = {}
```

```
for name, model in models.items():
    # Train
    model.fit(X_train_scaled, y_train)
    # Evaluate
    train_score = model.score(X_train_scaled, y_train)
    test_score = model.score(X_test_scaled, y_test)
    # Cross-validation
    cv_scores = cross_val_score(model, X_train_scaled, y_train, cv=5)
    results[name] = {
        'Train': train_score,
        'Test': test_score,
        'CV Mean': cv_scores.mean(),
        'CV Std': cv_scores.std()
    }
print(f"\n{name}:")
print(f" Training: {train_score:.3f}")
print(f" Testing: {test_score:.3f}")
print(f" CV Mean: {cv_scores.mean():.3f} (+/- {cv_scores.std():.3f})")
```

Model Comparison

- Visualizing Model Comparison

```
# Create comparison DataFrame
```

```
results_df = pd.DataFrame(results).T
```

```
# Plot comparison
```

```
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
```

```
# Bar plot of test scores
```

```
results_df["Test"].sort_values().plot(  
    kind='barh', ax=axes[0], color='skyblue'  
)
```

```
axes[0].set_xlabel("Test Accuracy")
```

```
axes[0].set_title("Model Test Performance")
```

```
axes[0].grid(axis='x', alpha=0.3)
```

```
# Train vs Test comparison
```

```
x = range(len(results_df))
```

```
width = 0.35
```

```
axes[1].bar([i - width/2 for i in x], results_df["Train"], width, label="Training",  
alpha=0.8)
```

```
axes[1].bar([i + width/2 for i in x], results_df["Test"], width, label="Testing",  
alpha=0.8)
```

```
axes[1].set_ylabel("Accuracy")
```

```
axes[1].set_title("Training vs Testing Performance")
```

```
axes[1].set_xticks(x)
```

```
axes[1].set_xticklabels(results_df.index, rotation=45, ha='right')
```

```
axes[1].legend()
```

```
axes[1].grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Identify best model
```

```
best_model = results_df["Test"].idxmax()
```

```
print(f"\n Best Model: {best_model}")
```

```
print(f"Test Accuracy: {results_df.loc[best_model, 'Test']:.3f}")
```